BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

CE-403: Database Administration (PostgreSQL)

Elchin Salahzada

Lecture 6

CE-403: Database Management Systems
Baku Higher Oil School : Computer Engineering

Indexes & Query Optimization

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- What is an Index & Why It Matters
- Types of Indexes (B-Tree, Hash, GIN, GiST)
- Creating & Managing Indexes in PostgreSQL
- Reading Query Execution Plans (EXPLAIN, EXPLAIN ANALYZE)
- Query Optimization Techniques
- Index Best Practices & Trade-offs
- Lab Activity

Learning objectives

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Explain what an index is and how it improves query performance
- Differentiate between index types (B-Tree, Hash, GIN, GiST) and identify appropriate use cases
- Create and manage indexes in PostgreSQL using CREATE INDEX and related commands
- Read and interpret query execution plans using EXPLAIN and EXPLAIN ANALYZE
- Apply query optimization techniques to improve slow-running queries
- Evaluate trade-offs of indexing (read performance vs. write overhead, storage cost)

None of opmitization

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

```
SELECT *
FROM customers c
JOIN orders o       ON o.customer_id = c.customer_id
JOIN order_items oi  ON oi.order_id   = o.order_id
WHERE LOWER(c.country) = 'azerbaijan'
  AND EXTRACT(YEAR FROM o.created_at) = 2025
  AND o.status IN (SELECT status FROM orders WHERE status <> 'cancelled')
  AND c.customer_id IN (
        SELECT customer_id FROM orders
GROUP BY customer_id HAVING SUM(total_amount) > 0
  )
ORDER BY (
    SELECT SUM(oi2.quantity * oi2.price)
    FROM order_items oi2
    JOIN orders o2 ON o2.order_id = oi2.order_id
    WHERE o2.customer_id = c.customer_id) DESC;
```

Problems

Problem	Effect
SELECT *	Reads and ships every column from three tables; blocks index-only scans
LOWER(c.country) = …	Function on the column makes a normal index unusable, so a sequential scan
EXTRACT(YEAR FROM o.created_at) = 2025	Not sargable: no index range scan on created_at
status IN (SELECT status FROM orders …)	Pointless self-subquery that scans orders again
customer_id IN (… GROUP BY … HAVING …)	Aggregates the entire orders table just to filter
Correlated subquery in ORDER BY	Runs once per row (N+1 pattern), re-joining two huge tables each time
No LIMIT	Sorts and returns the whole result set

Solution (create index)

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- -- Normalize the data once instead of calling LOWER() on every query
- -- (or store country as a code like 'AZ' in a dedicated column)
- CREATE INDEX idx_customers_country
- ON customers (country)
- INCLUDE (customer_id, full_name);
-
-
- -- Range filter on date + status, covering the columns the query needs
- CREATE INDEX idx_orders_created_status
- ON orders (created_at, status)
- INCLUDE (customer_id, order_id);
-
- -- Fast join from orders to items, covering the aggregated columns
- CREATE INDEX idx_order_items_order
- ON order_items (order_id)
- INCLUDE (quantity, price);

Solution (Optimized Code)

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

```
WITH az_customers AS (
   SELECT customer_id, full_name FROM customers  WHERE country = 'Azerbaijan' ),
year_orders AS (
  SELECT o.order_id, o.customer_id FROM orders o WHERE o.created_at >= TIMESTAMPTZ '2025-01-01'  AND o.created_at <  TIMESTAMPTZ '2026-01-01'
AND o.status <> 'cancelled')
SELECT c.customer_id, c.full_name,  COUNT(DISTINCT yo.order_id) AS order_count,
SUM(oi.quantity * oi.price) AS total_spent FROM az_customers c JOIN year_orders  yo
ON yo.customer_id = c.customer_id JOIN order_items  oi ON oi.order_id= yo.order_id
GROUP BY c.customer_id, c.full_name ORDER BY total_spent DESC LIMIT 100;
```

What is an Index

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- An index is a database object that improves the speed of data retrieval on a table
- Works similarly to a book's index — helps find data without scanning every page (row)
- Without an index: PostgreSQL performs a sequential scan — checks every row in the table
- With an index: PostgreSQL can jump directly to relevant rows, drastically reducing lookup time
- Trade-offs:
  - Speeds up SELECT queries (especially with WHERE, JOIN, ORDER BY)
  - Slows down INSERT, UPDATE, DELETE (index must be updated too)
  - Consumes additional storage space
- Indexes are most valuable on large tables and frequently queried columns

Types of Indexes

- PostgreSQL supports multiple index types, each optimized for different use cases
- B-Tree (default index type)
  - Best for equality and range queries (=, <, >, BETWEEN)
  - Used automatically for PRIMARY KEY and UNIQUE constraints
- Hash
  - Optimized for simple equality comparisons (=) only
  - Not suitable for range queries
- GIN (Generalized Inverted Index)
  - Best for indexing composite values — arrays, JSONB, full-text search
- GiST (Generalized Search Tree)
  - Supports complex data types — geometric data, full-text search, range types
- Choosing an index type depends on the data type and the kind of queries run against it

Creating & Managing Indexes

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Create a basic index (B-Tree by default):
  - CREATE INDEX idx_employees_name ON employees (name);
- Create an index with a specific type:
  - CREATE INDEX idx_employees_email ON employees USING HASH (email);
- Composite index (multiple columns):
  - CREATE INDEX idx_dept_salary ON employees (dept_id, salary);
- Unique index (enforces uniqueness):
  - CREATE UNIQUE INDEX idx_unique_email ON employees (email);
- List indexes on a table:
  - \di
- Drop an index:
  - DROP INDEX idx_employees_name;

Reading Query Execution Plans

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- PostgreSQL's query planner decides how to execute a query — EXPLAIN shows this plan
- EXPLAIN – shows the planned execution strategy (no actual execution)
  - EXPLAIN SELECT * FROM employees WHERE salary > 50000;
- EXPLAIN ANALYZE – actually runs the query and shows real execution time/stats
  - EXPLAIN ANALYZE SELECT * FROM employees WHERE salary > 50000;
- Key terms in output:
  - Seq Scan – full table scan (no index used)
  - Index Scan – index used to locate rows
  - Cost – estimated relative cost (startup cost..total cost)
  - Rows – estimated number of rows returned
  - Actual Time – real time taken (only with ANALYZE)
- Goal: identify Seq Scans on large tables and replace them with Index Scans where beneficial

Query Optimization Techniques

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Use indexes wisely – index columns used in WHERE, JOIN, and ORDER BY
- Avoid SELECT * – retrieve only the columns you need
- Use LIMIT when you don't need all rows
- Avoid functions on indexed columns in WHERE (e.g., WHERE UPPER(name) = 'JOHN' disables index use)
- Rewrite subqueries as JOINs where possible — often more efficient
- Use EXISTS instead of IN for large subquery result sets
- Analyze tables regularly – ANALYZE table_name; keeps query planner statistics up to date
- Avoid unnecessary DISTINCT or ORDER BY when not required
- Always test with EXPLAIN ANALYZE before and after optimization changes

Index Best Practices & Trade-offs

- Do:
  - Index columns frequently used in WHERE, JOIN, ORDER BY
  - Use composite indexes for queries filtering on multiple columns together
  - Monitor index usage with pg_stat_user_indexes
- Avoid:
  - Over-indexing — every index adds overhead to INSERT/UPDATE/DELETE
  - Indexing low-cardinality columns (e.g., boolean flags) — limited benefit
  - Redundant indexes covering the same columns
- Trade-offs:
  - Read performance ↑ vs. Write performance ↓
  - Storage cost increases with each additional index
- Rule of thumb: Index for your most frequent and performance-critical queries, not every column

Lecture Summary

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Indexes speed up data retrieval by avoiding full table scans, at the cost of write overhead and storage
- PostgreSQL supports multiple index types: B-Tree (default), Hash, GIN, GiST — each suited to different data/queries
- Indexes are created, managed, and dropped using CREATE INDEX, \di, and DROP INDEX
- EXPLAIN and EXPLAIN ANALYZE reveal how PostgreSQL executes a query — key signals: Seq Scan vs. Index Scan
- Optimization techniques include selective columns, avoiding functions on indexed columns, rewriting subqueries, and regular ANALYZE
- Best practice: index strategically — balance read performance gains against write overhead and storage cost

Hands-on

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Scenario: Optimize query performance on a larger employees dataset
- Insert a larger dataset into employees (e.g., generate 10,000+ rows using generate_series)
- Run a query filtering by salary and check its plan with EXPLAIN ANALYZE (observe Seq Scan)
- Create a B-Tree index on salary, re-run the same query, and compare the plan (observe Index Scan)
- Create a composite index on (dept_id, salary) and test a query filtering both columns
- Create a unique index on email and test constraint enforcement with a duplicate insert
- Run a query using UPPER(name) = 'ALICE' and observe that the index is not used — then rewrite the query to use the index effectively
- Drop one unnecessary/duplicate index and re-check \di
- Run ANALYZE employees; and re-test query plans

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

Next Lecture

- Stored Procedures, Functions & Triggers